

M

Act as a mid-level programmer in R.

Your job is to prepare documentation to set of functions in R which I am wrapping in my package.

Write documentation which will accompany the functions and will be recognised by roxygen2.

Write documentation to this method:

```
setClass("client4",
  slots = list(fname = "character",
    lname = "character",
    age = "numeric",
    gender = "character",
    married = "logical"),
  validity = function(object)
  {
    if(object@age < 0) return("Age cannot be negative!")
    if(object@age > 100) return("Age is too large!")
    if(!object@gender %in% c("F", "M"))
      return("Incorrect gender - use \"F\" or \"M\"!")
    return(TRUE)
  })
```



To create documentation for the "client4" class and its associated methods, you can use roxygen2 annotations. Here's an example of how you can write the documentation:



R

Copy code

```
#' client4 Class
#'
#' This class represents a client with basic personal information.
#'
#' @slot fname A character vector representing the first name of the client.
#' @slot lname A character vector representing the last name of the client.
#' @slot age A numeric vector representing the age of the client.
#' @slot gender A character vector representing the gender of the client. On
#' @slot married A logical vector indicating if the client is married (TRUE)
#'
#' @section Validity Method:
#'
#' @describeIn client4 The validity method for the client4 class.
#' This method checks the validity of the client's attributes.
#'
#' @param object An object of class "client4".
#'
```

```
#' @return A logical value indicating if the client's attributes are valid (
#' If the attributes are not valid, an error message is returned.
#'
#' @examples
#' client <- new("client4", fname = "John", lname = "Doe", age = 30, gender
#' validity(client)
#'
#' @export
setClass("client4",
  slots = list(
    fname = "character",
    lname = "character",
    age = "numeric",
    gender = "character",
    married = "logical"
  ),
  validity = function(object) {
    if (object@age < 0) return("Age cannot be negative!")
    if (object@age > 100) return("Age is too large!")
    if (!object@gender %in% c("F", "M"))
      return('Incorrect gender - use "F" or "M"!')
    return(TRUE)
  }
)
```

In the above example, the documentation is written using roxygen2 annotations. The `#'` symbol at the beginning of each line denotes a documentation line, and the `@` symbols are used for specific annotations.

Make sure to save this documentation in an R file with the same name as the function or class you are documenting, and include the roxygen2 package (`#' @import roxygen2`) in your package's DESCRIPTION file.

Writing prompts:

Writing prompts is an important skill which makes your AI usage much more efficient. The better your prompt, the higher your chances of getting a good and useful response. Remember about these key elements:

- Role - what role should the chat fulfill in the conversation:
 - Behave as an expert in creating content for Google positioning. Your role is to optimize the text to achieve the best positioning according to the latest guidelines (what are those guidelines), ensuring appropriate text density...
- Context - outline the context that the task pertains to - appropriately broad context:
 - I run a culinary blog about healthy eating and need to create a new blog post.
- Task - specify *precisely* what task the chat should perform:
 - Provide me with 10 inspirations for a new blog post on my blog.
- Conditions (optional) - additional conditions for ChatGPT to adhere to:
 - Exclude ideas for a post related to intermittent fasting because I have already written a lot about it recently.
- Response format (optional) - if you have a specific response format in mind (e.g., in the form of a table), specify it:
 - Organize the ideas in a table with columns: Number, Idea Title, Idea Description, Category (categories: food, dietary practices, others). Fill in each column.

Word of caution:

Never put any confidential data or company codes in the AI tools. It may be tempting, but we can never be sure where our input is actually used. This is a security threat.

AI can be a great help, but it is only a tool (and makes mistakes and wrong predictions like any other models you build).

Remember – it only predicts the most probable answer. If it has no knowledge about the subject – it can “hallucinate” (make up content just to fill the answer box).

Always take the AI output with a grain of salt. It is best to treat AI responses as a starting point, or an inspiration for your later work.

Train and learn how to optimize your own work with the assistance of the new tools, but don't depend on them too much. Treat it as the next search engine, instead of a big truthful oracle 😊

Never use it for writing thesis or big projects instead of you. It may seem like a convenient shortcut, but in truth it only makes you sidetrack. Mental exercise that you do today, will pay up in the future. You will never learn if you never take the actual effort to practice.

Have fun with exploring new products that come to the market. We live in very interesting and dynamic times. If you keep your head straight and develop critical thinking skills – you will always be on the winning side.